Software Requirements Specification

Project: Spotter

Version 2.0.1

Updated: October 29, 2026

Table of Contents

1. Introduction
    1.1 Purpose
    1.2 Scope
2. Overall Description
    2.1 User Classes and Characteristics
    2.2 Operating Environment
    2.3 Design and Implementation
    2.4 Assumptions and Dependencies
3. Functional Requirements
4. Non-Functional Requirements
5. Use Case Examples
6. Work Breakdown Structure
7. Timeline & Gantt Chart
8. Kanban/Scrum Board (Jira)
9. Risk, Quality, and Communication Management
    9.1 Risk Register
    9.2 Communication Plan
10. Roles and Resources
    10.1 Team Roles and Responsibilities
    10.2 Resource and Cost Plan
    10.3 RACI Matrix
11. Power-Interest Grid

1. Introduction

1.1 Purpose

The purpose of this document is to build an online software system that helps drivers locate, reserve, and pay for parking spaces and parking garages in real time. The platform will be available through web and mobile applications and will provide parking availability information, navigation assistance, and management tools for parking operators.

The goal is to develop a project management plan for the creation of this system, including requirements gathering, stakeholder management, scheduling, budgeting, risk management, and team coordination. Drivers often spend significant time searching for available parking, leading to congestion, frustration, and lost productivity. Parking operators need better tools to monitor occupancy, manage pricing, and improve utilization of parking assets.

1.2 Scope

1.2.1 Overview and In Scope

The service is a cross-platform parking application (iOS, Android, and web) that unifies two supply models under a single search-and-booking experience:

(Layer 1) Commercial Parking Booking: a reservation and payment system for professionally operated or municipally regulated parking spaces.

- User registration, authentication, login, and vehicle profile management
- Real-time interactive map-based search and price comparison across parking types: garages, lots, event/venue parking, airport parking, and metered/on-street parking
- Advance reservation and digital payment processing
- Digital pass generation (QR/barcode) for facility entry

- Session management to view, extend, or cancel an active/upcoming reservation
- Push/email notifications and alerts for session start, expiration, and reminders
- Reservation history, receipts, and saved/favorite locations
- Administrative dashboard and occupancy reporting and analytics for parking operators

(Layer 2) Peer-to-Peer Driveway Marketplace: a supplementary marketplace allowing homeowners near social hubs (arenas, airports, downtown districts) to list private driveway space for driver booking.

- Mockups/wireframes for host listing creation, driveway search results integrated into the main map view, booking confirmation, and a basic two-sided rating display
- Written user flow documentation describing how a host lists a space and how a driver discovers and books it
- No functional backend, payment processing, host verification system, or liability/insurance handling will be implemented for this layer in the current release; these are documented as future-phase requirements

1.2.2 Out of Scope

- Full host onboarding, identity verification, and payout processing
- Insurance/liability management or dispute resolution workflows
- Business/fleet parking account management
- In-car system integrations (e.g., CarPlay, Android Auto)
- AI-based customer support or chat features
- Real-time IoT sensor integration for live space availability

2. Overall Description

2.1 User Classes and Characteristics

Drivers (Primary Users)

Volume: Largest user group, majority of total active users

Role: Search for, reserve, and pay for parking across commercial (garage/lot/metered) and P2P driveway listings, manage active sessions

Technical proficiency: Basic smartphone/web literacy, no training required

Frequency of use: High/daily to weekly, depending on commute/travel habits

Access level: Standard registered-user permissions

Homeowner Hosts (P2P Supply Providers)

Volume: Small relative to Drivers, concentrated near target social hubs (arenas, airports, downtown districts)

Role: List private driveway/parking space, set availability and pricing; view incoming bookings

Technical proficiency: Basic, onboarding designed to require minimal setup effort

Frequency of use: Low to moderate, periodic listing updates rather than daily engagement

Access level: Host-tier permissions (listing management), no access to Driver-side booking tools beyond their own listings

Guest Users (Unregistered)

Volume: Variable, represents pre-conversion traffic (browsing before signup)

Role: Search and compare pricing only; cannot complete a reservation or payment

Technical proficiency: Basic

Frequency of use: Typically one-time or infrequent, until conversion to registered Driver

Access level: Restricted/read-only, until sign up

Parking Operators

Volume: Small, typically one operator account per managed facility, garage, or lot

Role: Monitor real-time occupancy, view reservation and revenue analytics, and adjust pricing for their managed facility/facilities via the operator dashboard

Technical proficiency: Basic to moderate, comfortable with standard business/web dashboard tools, no specialized training required

Frequency of use: Daily to weekly, operational check-ins rather than continuous engagement

Access level: Facility-scoped operator permissions = access limited to data and settings for facilities they manage. No visibility into other operators' data or Driver/Host account information

System Administrators

Volume: Very small, internal staff only

Role: Manage user accounts, resolve support issues, moderate host listings, monitor system health

Technical proficiency: Moderate to advanced, expected to use internal admin tooling

Frequency of use: Ongoing/operational

Access level: Elevated/administrative permissions

2.2 Operating Environment

Client Platforms

- Mobile: iOS and Android, delivered as native or cross-platform apps

- Web: Modern browsers (Chrome, Safari, Firefox, Edge) on desktop and mobile web

Server/Backend

- Cloud-hosted backend (e.g., AWS/Azure) using a RESTful API architecture
- Relational (MySQL) database for user, listing, and transaction data
- Standard load-balanced, horizontally scalable deployment for variable traffic

Third-Party Integrations

- Mapping/geolocation service provider for search, discovery, and directions
- Payment gateway provider for processing card and mobile wallet transactions
- Push notification service for session/reminder alerts
- (future phase) Identity verification service for Layer 2 host onboarding

Network Dependency

- Requires an active internet connection for all core functions; no offline mode is in scope

2.3 Design and Implementation Constraints

1. Fixed semester-long timeline
2. Dependence on third-party services and APIs
3. Compliance with data security and privacy requirements (PCI-DSS)
4. Platform pairing and consistent functionality across iOS/Android and web
5. Active network dependency
6. Variable volume scalability
7. Layer 2 features are constrained to mockup/prototype fidelity.

2.4 Assumptions and Dependencies

Assumptions

- Unlimited theoretical budget
- Users have access to a smartphone/web-enabled device and an active internet connection.
- Drivers and Hosts have a valid payment method and legal authority to list their property.
- Third-party mapping and payment services remain available, stable, and within their published API terms throughout development and testing.
- Layer 2 (P2P) functionality is assumed to remain at mockup/prototype for the duration of this project and will not require live transaction support.
- Local zoning/HOA restrictions on residential parking rentals are assumed to be a future-phase concern, not something the current system needs to enforce or validate.

Dependencies

- System functionality depends on the continued availability of the chosen mapping/geolocation service provider.
- Payment processing depends on a third-party payment gateway and its compliance infrastructure.
- Push notifications depend on platform-level notification services (e.g., APNs, FCM) for mobile, and browser notification APIs for web.
- Cross-platform consistency depends on a shared backend/API layer being implemented before mobile and web clients are built out.
- Any future expansion of Layer 2 to full functionality depends on adding identity verification and payout processing services not currently in scope.

3. Functional Requirements

FR1: The system shall allow users to register, log in, and manage a vehicle profile.

FR2: The system shall allow users to search for parking by location and time, displaying results on a map view.

FR3: The system shall display available parking types with associated pricing.

FR4: The system shall allow users to compare prices across nearby parking options before booking.

FR5: The system shall allow users to reserve and prepay for a parking session via credit/debit card or mobile wallet.

FR6: The system shall generate a digital pass (QR/barcode) upon successful booking for entry.

FR7: The system shall allow users to view, extend, or cancel an active or upcoming reservation.

FR8: The system shall send notifications for session start, upcoming expiration, and reminders.

FR9: The system shall maintain a booking history/receipts and allow users to save favorite locations.

FR10: The system shall display prototype-level P2P driveway listings (mockup data) integrated into the main search/map view.

FR11: The system shall provide a mockup host listing creation flow (non-functional data entry, no live backend persistence required).

FR12: The system shall display a mockup two-sided rating/review interface for Layer 2 bookings.

FR13: The system shall provide parking operators with a dashboard displaying real-time occupancy for their managed facility/facilities.

FR14: The system shall allow parking operators to view reservation and revenue history for their facility over a selectable date range.

FR15: The system shall allow parking operators to view and adjust pricing for their listed parking spaces, subject to system-defined limits.

FR16: The system shall generate basic analytics (e.g., average occupancy rate, peak usage hours, total reservations) for a parking operator's facility.

FR17: The system shall restrict dashboard access to authenticated parking operator accounts, limited to data for facilities they manage.

4. Non-Functional Requirements

NFR1: The system shall be available on iOS/Android and web platforms with consistent core functionality.

NFR2: The system shall respond to search queries within 2 seconds under normal load conditions.

NFR3: The system shall scale horizontally to handle demand spikes during high-traffic events.

NFR4: The system shall encrypt all payment and personal data in transit and at rest, complying with PCI-DSS standards.

NFR5: The system shall maintain 99% uptime for core booking functionality during business/operational hours.

NFR6: The system's UI shall be usable by individuals with basic smartphone/web literacy.

NFR7: The system shall log and handle failures from internal mapping and payment service abstractions gracefully, without exposing raw third-party errors to end users.

NFR8: The system shall support at least the two most recent major OS versions for iOS/Android, and current web browsers.

NFR9: The system shall not require offline functionality, but shall fail with a clear error statement when no network connection is present.

NFR10: The system's architecture shall allow Layer 2 to be extended from prototype to full functionality without requiring a redesign of the shared backend/data model.

NFR11: The operator dashboard shall reflect occupancy data with no more than a 1-minute delay from the underlying reservation system.

NFR12: The operator dashboard shall be accessible via web only, consistent with an internal/business-tool use case rather than a consumer-facing feature.

NFR13: Analytics data displayed on the dashboard shall be scoped and access-controlled so that one operator cannot view another operator's facility data.

5. Use Case Examples

Register Account
Actor: Driver, Host
Precondition: User has downloaded the app or accessed the web platform and is not yet registered
Steps:

1. User selects "Sign Up"
2. User enters name, email, and password
3. System validates input and creates account
4. System sends confirmation notification

Postcondition: User account is created and user is logged in

Log In
Actor: Driver, Host, Administrator, Parking Operator
Precondition: User has a registered account
Steps:

1. User enters email and password
2. System validates credentials
3. System grants access to the appropriate dashboard based on user role

Postcondition: User is authenticated and directed to their home screen

Guest Browses Parking Availability

Actor: Guest User
Precondition: User has not logged in or registered
Steps:

1. Guest opens the app/web platform
2. Guest enters a location, date, and timeframe
3. System displays available parking options and pricing on a map
4. Guest attempts to book and is prompted to register/log in

Postcondition: Guest views available parking but cannot complete a reservation

Manage Vehicle Profile
Actor: Driver
Precondition: Driver is logged in
Steps:

1. Driver navigates to profile settings
2. Driver adds or edits vehicle details (license plate, vehicle type)
3. System saves updated profile

Postcondition: Vehicle profile is updated and available for future bookings

Search for Parking
Actor: Driver
Precondition: Driver is logged in
Steps:

1. Driver enters a destination or enables location services
2. System queries available parking spaces (garages, lots, metered, event/airport)
3. System displays results on a map view

Postcondition: Driver views a list/map of nearby available parking options

Compare Parking Prices
Actor: Driver
Precondition: Search results have been returned
Steps:

1. Driver views multiple parking options in the results list
2. System displays address, price, distance, walking time, and duration for each option
3. Driver selects a preferred option

Postcondition: Driver has identified a chosen parking option to book

Reserve and Pay for Parking
Actor: Driver
Precondition: Driver has selected a parking option and is logged in
Steps:

1. Driver confirms reservation start/end time
2. Driver confirms vehicle profile and payment method
3. System processes payment through the payment service
4. System confirms the reservation

Postcondition: Reservation is created and payment is processed
* payment failure is handled in “Handle Declined Payment”

View Digital Pass for Entry
Actor: Driver
Precondition: Driver has an active reservation
Steps:

1. Driver opens the reservation from their booking list
2. System displays a QR code/barcode digital pass
3. Driver presents the pass at the facility entrance

Postcondition: Driver gains entry to the reserved parking space

Extend Active Parking Session
Actor: Driver
Precondition: Driver has an active, ongoing reservation
Steps:

1. Driver selects "Extend Session" from the active reservation screen.
2. Driver selects additional time interval (e.g., add 1hr for $1.50, add 2hrs for $2.00)
3. System checks availability and processes additional payment
4. System updates the reservation end time

Postcondition: Reservation duration is extended

Cancel Reservation
Actor: Driver

Precondition: Driver has an upcoming or active reservation
Steps:

1. Driver selects "Cancel" on the reservation
2. System displays cancellation terms (e.g., refund eligibility)
3. Driver confirms cancellation
4. System processes refund if applicable and cancels the reservation

Postcondition: Reservation is canceled and reflected in booking history

Receive Session Notification
Actor: Driver
Precondition: Driver has an active or upcoming reservation
Steps:

1. System monitors reservation status and time remaining
2. System triggers a notification at user-defined intervals (e.g, 10 minutes before reservation time)
3. Driver receives push/email notification

Postcondition: Driver is informed of session status in a timely manner
* timing is set in “Manage Notification Preferences”

View Booking History/Receipts and Manage Favorites
Actor: Driver
Precondition: Driver is logged in and has at least one past reservation
Steps:

1. Driver navigates to "Booking History"
2. System displays past and current reservations with payment history
3. Driver selects a location and marks it as a favorite

Postcondition: Location is saved for faster future booking

Browse P2P Driveway Listings (Prototype)
Actor: Driver
Precondition: Driver is logged in and searching in an area with prototype Layer 2 listings
Steps:

1. Driver's search results include driveway listings alongside commercial options
2. Driver selects a driveway listing to view mockup details (photos, price, availability)
3. Driver views a mockup booking confirmation screen (non-functional)

Postcondition: Driver views prototype P2P booking flow; no live transaction occurs

Create Host Listing (Prototype)
Actor: Host
Precondition: Host is logged in
Steps:

1. Host selects "List My Lot"
2. Host enters mockup listing details (address, photos, price, availability window, etc.)
3. System displays a confirmation screen showing the listing preview

Postcondition: A prototype listing preview is generated, no live persistence or public listing occurs

Administrator Moderates Host Listing / Resolves Dispute
Actor: Administrator
Precondition: Administrator is logged in with elevated permissions
Steps:

1. Administrator reviews flagged host listings or user disputes
2. Administrator reviews relevant booking/account data
3. Administrator approves, removes, or escalates the listing/dispute

Postcondition: Listing status is updated or dispute is resolved/escalated

View Facility Occupancy Dashboard
Actor: Parking Operator
Precondition: Operator is logged in and manages at least one facility in the system
Steps:

1. Operator logs in and navigates to the Dashboard
2. System retrieves current occupancy data for the operator's facility/facilities
3. System displays occupancy as a percentage or count (spaces filled/available)

Postcondition: Operator views real-time occupancy status for their facility

View Reservation and Revenue Analytics
Actor: Parking Operator
Precondition: Operator is logged in and has historical reservation data
Steps:

1. Operator navigates to the Analytics section of the dashboard
2. Operator selects a date range
3. System displays reservation counts, revenue totals, and peak usage hours for that range

Postcondition: Operator views summarized analytics for the selected period.

Adjust Facility Pricing
Actor: Parking Operator
Precondition: Operator is logged in and manages a facility with existing pricing data
Steps:

1. Operator navigates to facility settings
2. Operator updates pricing for one or more parking types (hourly, daily, event)
3. System validates the input against defined pricing limits
4. System saves and applies the updated pricing to future searches/bookings

Postcondition: Updated pricing is reflected in the system for new reservations

Reset Password
Actor: Driver, Host, Parking Operator, Administrator
Precondition: User has a registered account but cannot log in
Steps:

1. User selects “Forgot Password” on the login screen
2. User enters their registered email
3. System sends a time-limited reset link
4. User opens the link and enters a new password

5. System validates the password and updates the account

Postcondition: Password is updated and the user can log in with the new credentials

Restrict Operator Access to Own Facilities
Actor: Parking Operator
Precondition: Operator is logged in and manages at least one facility
Steps:

1. Operator attempts to open data for a facility they do not manage
2. System checks the operator's facility-scoped permissions
3. System denies access and shows an authorization message without revealing any facility data
4. System logs the attempt

Postcondition: No data from other facilities is exposed and the attempt is recorded

Manage Saved Payment Methods
Actor: Driver
Precondition: Driver is logged in
Steps:

1. Driver navigates to "Payment Methods" in profile settings
2. Driver adds a card or mobile wallet
3. System sends the details to the payment service for validation (card data is not stored by Spotter)
4. Driver sets a default method or removes an existing one

Postcondition: Saved methods are available for selection during booking

Handle Declined Payment
Actor: Driver
Precondition: Driver has submitted payment for a reservation
Steps:

1. Payment service returns a decline
2. System displays a clear error message without exposing raw gateway details
3. System preserves the driver's booking selection
4. Driver retries or chooses a different payment method

Postcondition: Either the reservation is confirmed after successful payment, or no reservation or charge is created

Manage Notification Preferences
Actor: Driver, Host
Precondition: User is logged in
Steps:

1. User opens notification settings
2. User selects channels (push, email) and reminder timing (e.g., 10 or 30 minutes before expiration)
3. User saves changes
4. System applies the preferences to future sessions

Postcondition: Preferences are saved and used by "Receive Session Notification"

Manage Facility Settings

Actor: Parking Operator
Precondition: Operator is logged in and manages a facility
Steps:

1. Operator opens facility settings
2. Operator edits details such as operating hours, listed capacity, or entry instructions
3. System validates the input
4. System saves the changes and applies them to the facility's listing

Postcondition: Updated settings appear in search results and booking details.

Manage User Accounts
Actor: Administrator
Precondition: Administrator is logged in with elevated permissions
Steps:

1. Administrator searches for a user account
2. Administrator reviews account status and recent activity
3. Administrator suspends, reactivates, or resets access for the account
4. System applies the change and records it in the audit log

Postcondition: Account status is updated and the action is logged

Export Operator Report
Actor: Parking Operator
Precondition: Operator is logged in and has reservation data for their facility
Steps:

1. Operator opens the Reports section
2. Operator selects a report type (occupancy, revenue, reservation count, peak hours) and a date range
3. Operator chooses CSV or PDF
4. System generates the file using only the operator's own facility data
5. Operator downloads the file

Postcondition: Report file is downloaded

Rate and Review a Completed Booking (Prototype)
Actor: Driver, Host
Precondition: A mockup completed P2P booking exists
Steps:

1. User opens the completed booking
2. User selects a star rating and enters an optional comment
3. User submits
4. System displays a mockup confirmation and the two-sided rating display

Postcondition: Prototype rating is shown; no live persistence occurs

Monitor System Health
Actor: Administrator

Precondition: Administrator is logged in with elevated permissions
Steps:

1. Administrator opens the System Health view
2. System displays service status (API, database, mapping, payment), error rates, and response times
3. Administrator reviews active alerts
4. Administrator acknowledges an alert or escalates it to DevOps

Postcondition: Alerts are acknowledged or escalated and the review is logged

Handle Network or Service Failure
Actor: Driver, Host, Parking Operator
Precondition: User is using the app when connectivity or a mapping/payment service becomes unavailable
Steps:

1. System detects the failed request or lost connection
2. System shows a clear error with a retry option and no raw third-party error text
3. System logs the failure
4. User retries, or the system resumes automatically once service returns, preserving entered booking details

Postcondition: User is informed and no partial or duplicate booking or charge is created

6. Work Breakdown Structure (App Functionalities)

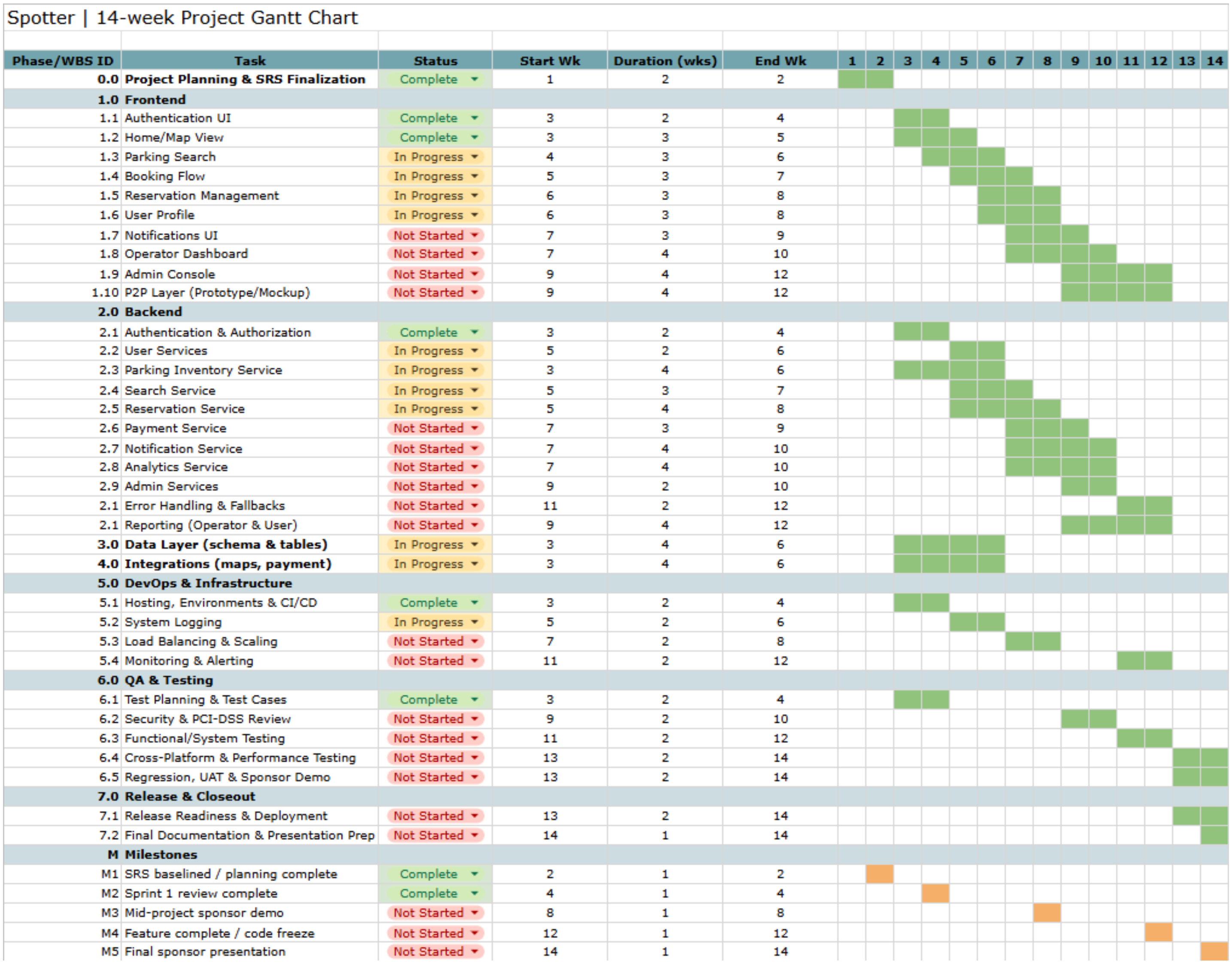
- 0.0 Project Planning & SRS Finalization [NEW]
    - 0.1 SRS drafting and baseline
    - 0.2 Planning artifacts (WBS, Gantt, risk register, communication plan, cost plan, RACI)
- 1.0 Frontend
    - 1.1 Authentication UI
        - 1.1.1 Sign-up form
        - 1.1.2 Login form
        - 1.1.3 Password reset
        - 1.1.4 Role-based dashboard redirect (Driver/Host/Operator/Admin)
    - 1.2 Home/Map View
        - 1.2.1 Interactive map render
        - 1.2.2 Location permission/geolocation prompt
        - 1.2.3 Navigation bar and menu
        - 1.2.4 Guest vs logged-in state handling
    - 1.3 Parking Search
        - 1.3.1 Destination input (autocomplete)
        - 1.3.2 Date/time selector
        - 1.3.3 Results list view
        - 1.3.4 Map pin rendering (commercial + P2P listing types)
        - 1.3.5 Filters (price, distance, parking type)
        - 1.3.6 Listing detail view (address, price, walking time, photos)
    - 1.4 Booking Flow
        - 1.4.1 Reservation day/time confirmation
        - 1.4.2 Vehicle profile selection
        - 1.4.3 Payment method selection
        - 1.4.4 Booking confirmation screen
        - 1.4.5 Declined-payment error and retry flow [NEW]
    - 1.5 Reservation Management
        - 1.5.1 Active/upcoming reservation list
        - 1.5.2 Digital pass display (QR/barcode)
        - 1.5.3 Extend session UI
        - 1.5.4 Cancel reservation UI with terms display
    - 1.6 User Profile
        - 1.6.1 Account settings
        - 1.6.2 Vehicle profile add/edit
        - 1.6.3 Saved payment methods (add, remove, set default)
        - 1.6.4 Booking history and receipts
        - 1.6.5 Favorite locations
    - 1.7 Notifications UI
        - 1.7.1 In-app notification display
        - 1.7.2 Notification preference settings (channels and reminder timing)
    - 1.8 Operator Dashboard (web only)
        - 1.8.1 Occupancy display
        - 1.8.2 Analytics view (date range selector, charts)

        - 1.8.3 Pricing adjustment form
        - 1.8.4 Facility settings (hours, capacity, entry instructions)
        - 1.8.5 Report request and export UI (CSV/PDF) [NEW]
    - 1.9 Admin Console
        - 1.9.1 User account management view
        - 1.9.2 Flagged listing and dispute queue
        - 1.9.3 System health monitoring view
        - 1.9.4 Admin console layout and navigation [NEW]
    - 1.10 P2P Layer (Prototype/Mockup)
        - 1.10.1 Host listing creation screens
        - 1.10.2 Driveway listing detail mockup
        - 1.10.3 Mockup booking confirmation
        - 1.10.4 Two-sided rating/review display
        - 1.10.5 P2P map search integration mockup [NEW]
        - 1.10.6 P2P user flow documentation (host listing and driver booking) [NEW]
- 2.0 Backend
    - 2.1 Authentication & Authorization
        - 2.1.1 User registration service
        - 2.1.2 Login/session token handling
        - 2.1.3 Role-based access control (Driver/Host/Operator/Admin)
        - 2.1.4 Facility-scoped operator permissions
    - 2.2 User Services
        - 2.2.1 Profile CRUD
        - 2.2.2 Vehicle profile CRUD
        - 2.2.3 Favorites management
        - 2.2.4 Booking history retrieval [NEW]
    - 2.3 Parking Inventory Service
        - 2.3.1 Facility/space data model
        - 2.3.2 Availability lookup
        - 2.3.3 Pricing rules engine
        - 2.3.4 Parking type handling (garage, lot, metered, event, airport)
    - 2.4 Search Service
        - 2.4.1 Location-based query handling
        - 2.4.2 Results ranking/sorting
        - 2.4.3 Price comparison logic
    - 2.5 Reservation Service
        - 2.5.1 Create reservation
        - 2.5.2 Extend reservation
        - 2.5.3 Cancel reservation and refund eligibility
        - 2.5.4 Reservation state management
        - 2.5.5 Digital pass generation (QR/barcode)
    - 2.6 Payment Service
        - 2.6.1 Payment gateway integration layer
        - 2.6.2 Transaction processing
        - 2.6.3 Refund processing
        - 2.6.4 Receipt generation

        - 2.6.5 Payment failure handling (declined payment) [NEW]
        - 2.6.6 Saved payment method management (card data not stored by Spotter) [NEW]
    - 2.7 Notification Service
        - 2.7.1 Scheduled session reminders
        - 2.7.2 Push dispatch (APNs/FCM)
        - 2.7.3 Email dispatch
        - 2.7.4 Notification preference handling [NEW]
    - 2.8 Analytics Service
        - 2.8.1 Occupancy aggregation
        - 2.8.2 Reservation/revenue aggregation [RENAMED]
        - 2.8.3 Peak usage calculation
    - 2.9 Admin Services
        - 2.9.1 Account moderation actions
        - 2.9.2 Listing moderation
        - 2.9.3 Dispute record handling
        - 2.9.4 Audit logging of admin actions [NEW]
    - 2.10 Error Handling & Fallbacks [RENAMED]
        - 2.10.1 Third-party failure handling/fallbacks
        - 2.10.2 Offline/no-network error states [RENUMBERED]
    - 2.11 Reporting Service (Operator & User) [NEW]
        - 2.11.1 Operator reports (occupancy, revenue, reservation count, peak hours)
        - 2.11.2 Report export (CSV/PDF)
        - 2.11.3 User reports (booking history, payments/receipts, favorites summary, cancellations/refunds)
        - 2.11.4 Session notification log
- 3.0 Data Layer
    - 3.1 User & account tables
    - 3.2 Vehicle profile tables
    - 3.3 Facility/parking space tables
    - 3.4 Reservation tables
    - 3.5 Transaction/payment records
    - 3.6 Analytics/occupancy records
    - 3.7 P2P listing schema (future phase; delivered with 1.10)
- 4.0 Integrations [RENAMED]
    - 4.1 Mapping/geolocation service
    - 4.2 Payment gateway (sandbox setup)
    - 4.3 Push notification service (connected with 2.7.2)
    - 4.4 Identity verification (future phase)
- 5.0 DevOps & Infrastructure [NEW]
    - 5.1 Hosting, Environments & CI/CD
        - 5.1.1 Cloud hosting and environment setup (dev/staging/prod)
        - 5.1.2 CI/CD pipeline
    - 5.2 System Logging [MOVED]
        - 5.2.1 Centralized application and API logging (formerly 2.10.2)
    - 5.3 Load Balancing & Scaling

    - 5.3.1 Load balancer configuration
    - 5.3.2 Horizontal scaling rules for event-driven demand spikes
  - 5.4 Monitoring & Alerting
    - 5.4.1 Service health monitoring (feeds 1.9.3)
    - 5.4.2 Alert rules and escalation
- 6.0 QA & Testing [NEW]
  - 6.1 Test Planning & Test Cases
    - 6.1.1 Test plan
    - 6.1.2 Test case authoring from FRs, NFRs and use cases
  - 6.2 Security & PCI-DSS Review
    - 6.2.1 Payment and personal data handling review
    - 6.2.2 Access control and facility-scope testing
  - 6.3 Functional/System Testing
    - 6.3.1 End-to-end functional testing
    - 6.3.2 Defect logging
  - 6.4 Cross-Platform & Performance Testing
    - 6.4.1 iOS, Android and web compatibility testing
    - 6.4.2 Load and response-time testing
  - 6.5 Regression, UAT & Sponsor Demo
    - 6.5.1 Defect triage and regression testing
    - 6.5.2 Operator user acceptance testing
    - 6.5.3 Sponsor demo
- 7.0 Release & Closeout [NEW]
  - 7.1 Release Readiness & Deployment
    - 7.1.1 Release checklist and go/no-go decision
    - 7.1.2 Production deployment
  - 7.2 Final Documentation & Presentation Prep [RENUMBERED]
    - 7.2.1 Final SRS and project documentation updates
    - 7.2.2 Final presentation

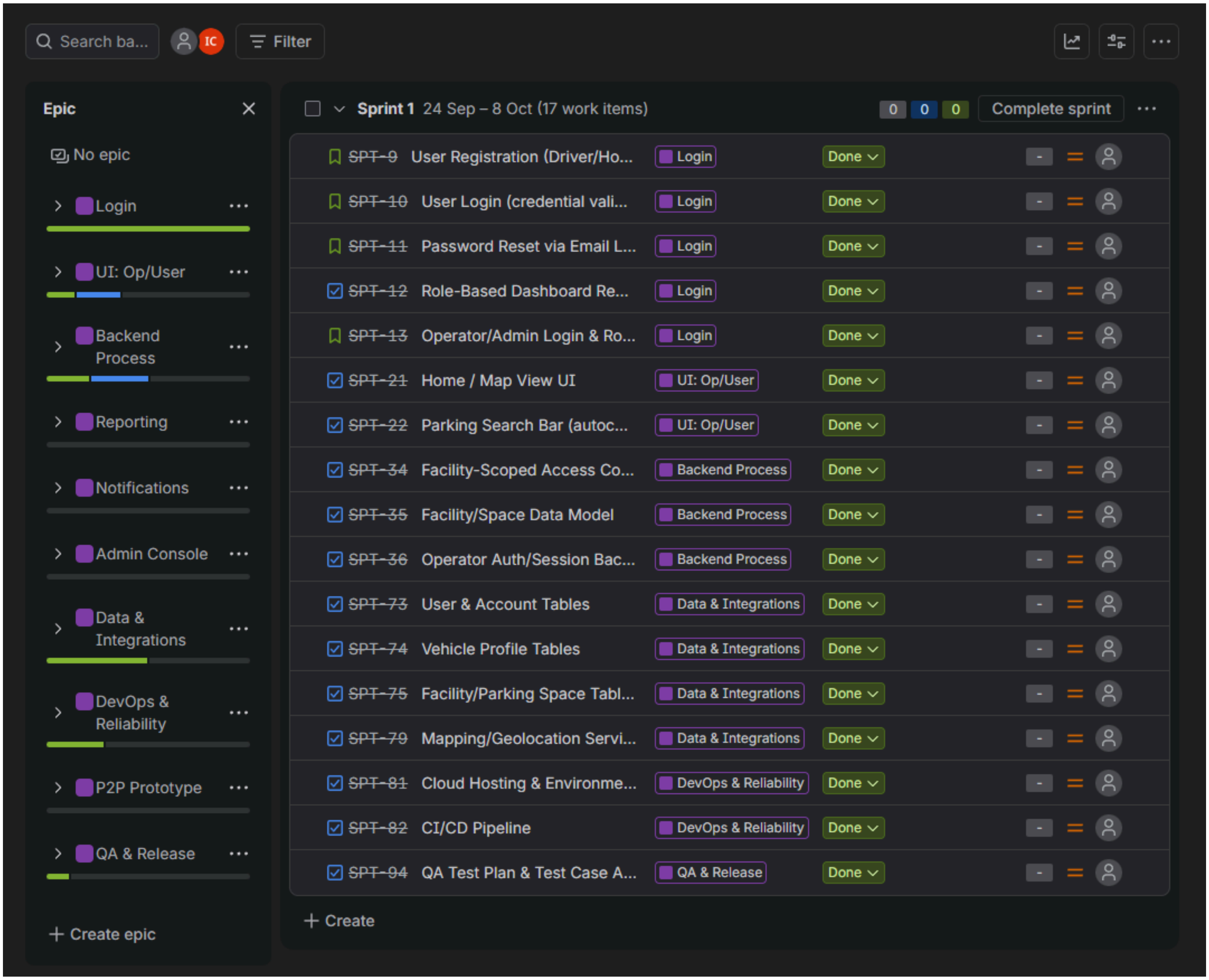
7. Timeline & Gantt Chart



Spotter | 14-week Project Gantt Chart

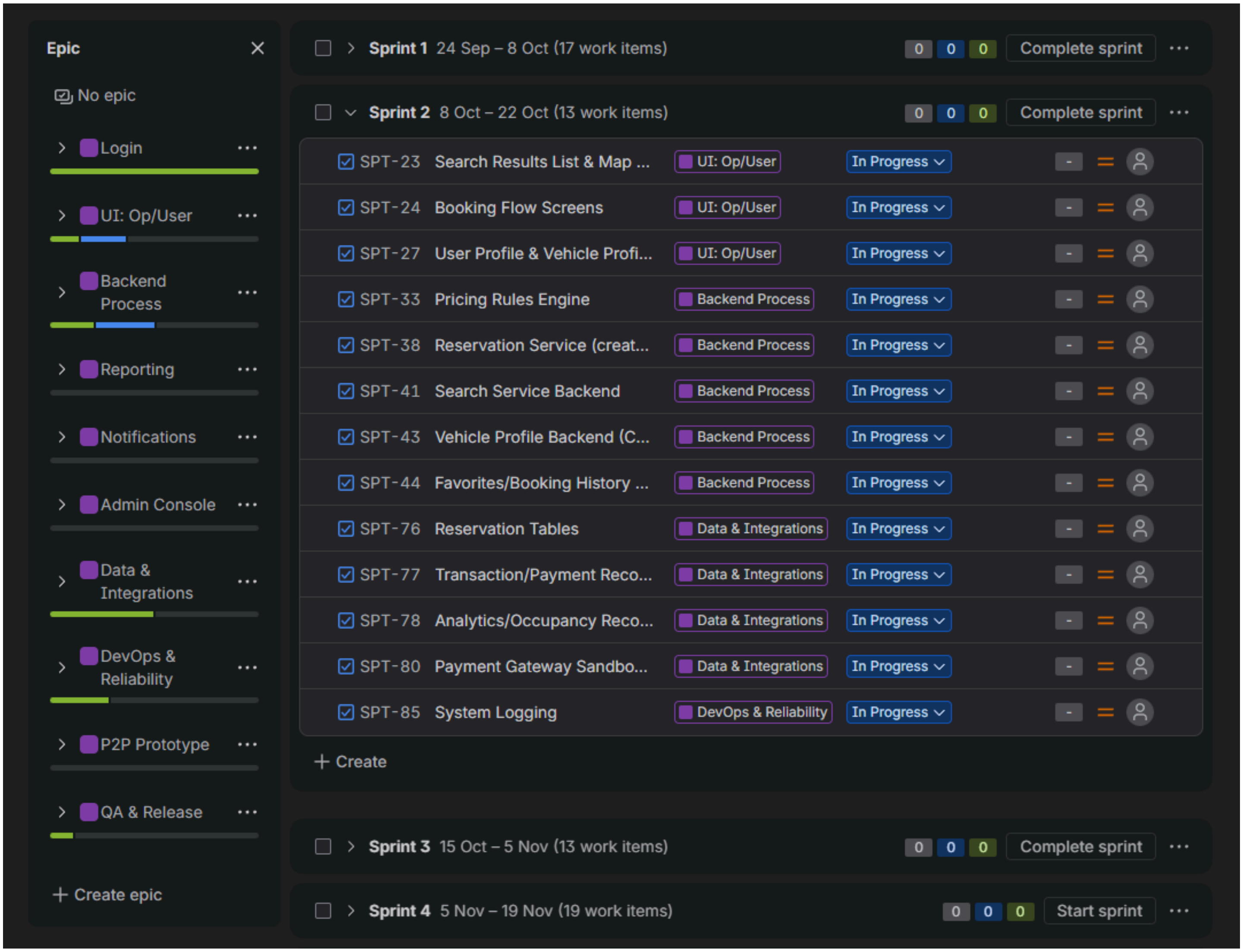
Phase/WBS ID	Task	Status	Start Wk	Duration (wks)	End Wk	1	2	3	4	5	6	7	8	9	10	11	12	13	14
0.0	**Project Planning & SRS Finalization**	Complete	1	2	2	🟩	🟩												
1.0	**Frontend**																		
1.1	Authentication UI	Complete	3	2	4			🟩	🟩										
1.2	Home/Map View	Complete	3	3	5			🟩	🟩	🟩									
1.3	Parking Search	In Progress	4	3	6				🟩	🟩	🟩								
1.4	Booking Flow	In Progress	5	3	7					🟩	🟩	🟩							
1.5	Reservation Management	In Progress	6	3	8						🟩	🟩	🟩						
1.6	User Profile	In Progress	6	3	8						🟩	🟩	🟩						
1.7	Notifications UI	Not Started	7	3	9							🟩	🟩	🟩					
1.8	Operator Dashboard	Not Started	7	4	10							🟩	🟩	🟩	🟩				
1.9	Admin Console	Not Started	9	4	12									🟩	🟩	🟩	🟩		
1.10	P2P Layer (Prototype/Mockup)	Not Started	9	4	12									🟩	🟩	🟩	🟩		
2.0	**Backend**																		
2.1	Authentication & Authorization	Complete	3	2	4			🟩	🟩										
2.2	User Services	In Progress	5	2	6					🟩	🟩								
2.3	Parking Inventory Service	In Progress	3	4	6			🟩	🟩	🟩	🟩								
2.4	Search Service	In Progress	5	3	7					🟩	🟩	🟩							
2.5	Reservation Service	In Progress	5	4	8					🟩	🟩	🟩	🟩						
2.6	Payment Service	Not Started	7	3	9							🟩	🟩	🟩					
2.7	Notification Service	Not Started	7	4	10							🟩	🟩	🟩	🟩				
2.8	Analytics Service	Not Started	7	4	10							🟩	🟩	🟩	🟩				
2.9	Admin Services	Not Started	9	2	10									🟩	🟩				
2.1	Error Handling & Fallbacks	Not Started	11	2	12											🟩	🟩		
2.1	Reporting (Operator & User)	Not Started	9	4	12									🟩	🟩	🟩	🟩		
3.0	**Data Layer (schema & tables)**	In Progress	3	4	6			🟩	🟩	🟩	🟩								
4.0	**Integrations (maps, payment)**	In Progress	3	4	6			🟩	🟩	🟩	🟩								
5.0	**DevOps & Infrastructure**																		
5.1	Hosting, Environments & CI/CD	Complete	3	2	4			🟩	🟩										
5.2	System Logging	In Progress	5	2	6					🟩	🟩								
5.3	Load Balancing & Scaling	Not Started	7	2	8							🟩	🟩						
5.4	Monitoring & Alerting	Not Started	11	2	12											🟩	🟩		
6.0	**QA & Testing**																		
6.1	Test Planning & Test Cases	Complete	3	2	4			🟩	🟩										
6.2	Security & PCI-DSS Review	Not Started	9	2	10									🟩	🟩				
6.3	Functional/System Testing	Not Started	11	2	12											🟩	🟩		
6.4	Cross-Platform & Performance Testing	Not Started	13	2	14													🟩	🟩
6.5	Regression, UAT & Sponsor Demo	Not Started	13	2	14													🟩	🟩
7.0	**Release & Closeout**																		
7.1	Release Readiness & Deployment	Not Started	13	2	14													🟩	🟩
7.2	Final Documentation & Presentation Prep	Not Started	14	1	14														🟩
M	**Milestones**																		
M1	SRS baselined / planning complete	Complete	2	1	2		🟧												
M2	Sprint 1 review complete	Complete	4	1	4				🟧										
M3	Mid-project sponsor demo	Not Started	8	1	8								🟧						
M4	Feature complete / code freeze	Not Started	12	1	12												🟧		
M5	Final sponsor presentation	Not Started	14	1	14														🟧

8. Kanban/Scrum Board (Jira)


Search ba...
IC
Filter
Epic
No epic
Login
UI: Op/User
Backend Process
Reporting
Notifications
Admin Console
Data & Integrations
DevOps & Reliability
P2P Prototype
QA & Release
Create epic
0
0
0
Complete sprint
Create

Sprint 1 24 Sep – 8 Oct (17 work items)

~~SPT-9~~	User Registration (Driver/Ho...	Login	Done
~~SPT-10~~	User Login (credential vali...	Login	Done
~~SPT-11~~	Password Reset via Email L...	Login	Done
~~SPT-12~~	Role-Based Dashboard Re...	Login	Done
~~SPT-13~~	Operator/Admin Login & Ro...	Login	Done
~~SPT-21~~	Home / Map View UI	UI: Op/User	Done
~~SPT-22~~	Parking Search Bar (autoc...	UI: Op/User	Done
~~SPT-34~~	Facility-Scoped Access Co...	Backend Process	Done
~~SPT-35~~	Facility/Space Data Model	Backend Process	Done
~~SPT-36~~	Operator Auth/Session Bac...	Backend Process	Done
~~SPT-73~~	User & Account Tables	Data & Integrations	Done
~~SPT-74~~	Vehicle Profile Tables	Data & Integrations	Done
~~SPT-75~~	Facility/Parking Space Tabl...	Data & Integrations	Done
~~SPT-79~~	Mapping/Geolocation Servi...	Data & Integrations	Done
~~SPT-81~~	Cloud Hosting & Environme...	DevOps & Reliability	Done
~~SPT-82~~	CI/CD Pipeline	DevOps & Reliability	Done
~~SPT-94~~	QA Test Plan & Test Case A...	QA & Release	Done


Epic
No epic
Login
UI: Op/User
Backend Process
Reporting
Notifications
Admin Console
Data & Integrations
DevOps & Reliability
P2P Prototype
QA & Release
+ Create epic

☐ Sprint 1 24 Sep – 8 Oct (17 work items) 0 0 0 Complete sprint

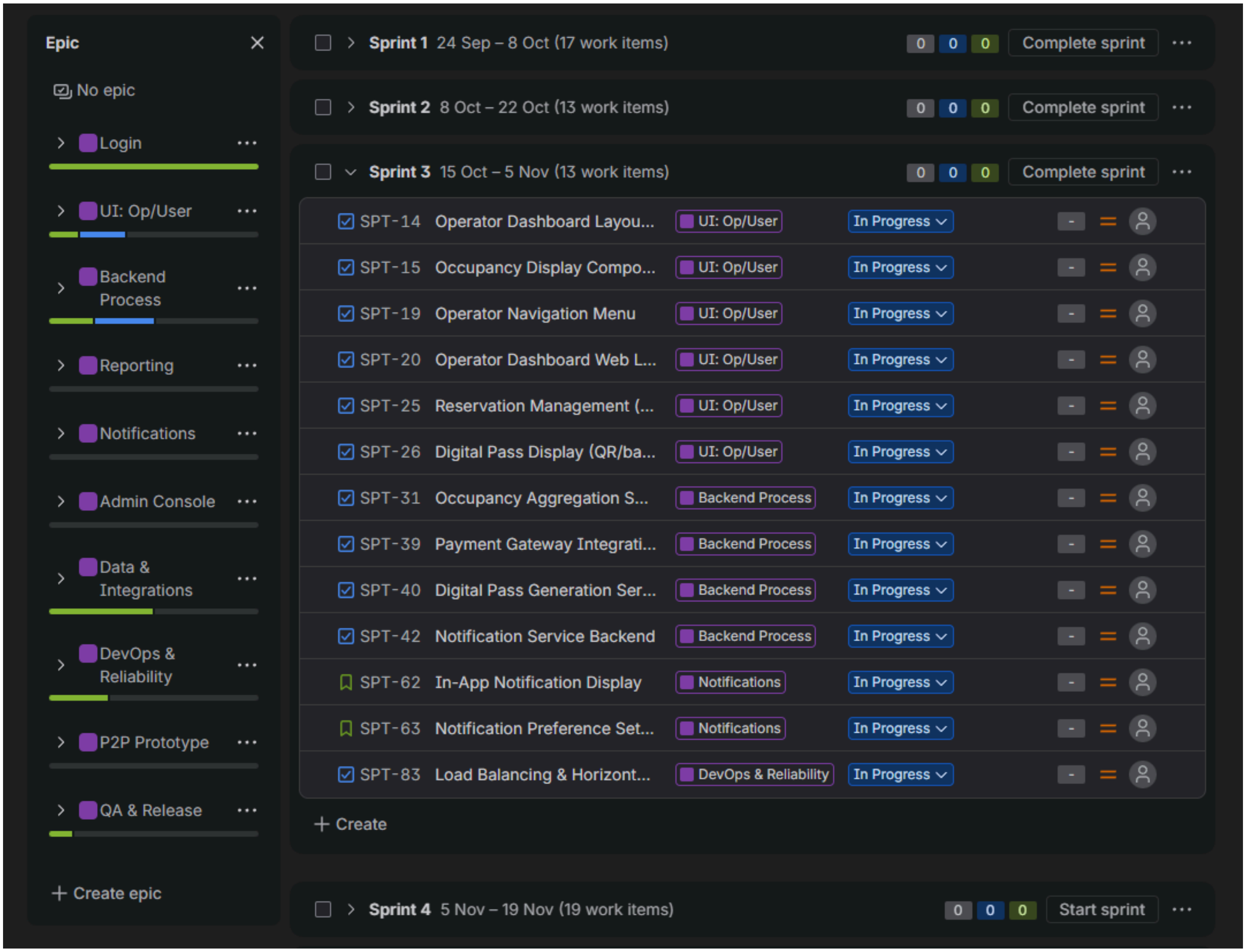
☐ Sprint 2 8 Oct – 22 Oct (13 work items) 0 0 0 Complete sprint

☑ SPT-23	Search Results List & Map ...	UI: Op/User	In Progress
☑ SPT-24	Booking Flow Screens	UI: Op/User	In Progress
☑ SPT-27	User Profile & Vehicle Profi...	UI: Op/User	In Progress
☑ SPT-33	Pricing Rules Engine	Backend Process	In Progress
☑ SPT-38	Reservation Service (creat...	Backend Process	In Progress
☑ SPT-41	Search Service Backend	Backend Process	In Progress
☑ SPT-43	Vehicle Profile Backend (C...	Backend Process	In Progress
☑ SPT-44	Favorites/Booking History ...	Backend Process	In Progress
☑ SPT-76	Reservation Tables	Data & Integrations	In Progress
☑ SPT-77	Transaction/Payment Reco...	Data & Integrations	In Progress
☑ SPT-78	Analytics/Occupancy Reco...	Data & Integrations	In Progress
☑ SPT-80	Payment Gateway Sandbo...	Data & Integrations	In Progress
☑ SPT-85	System Logging	DevOps & Reliability	In Progress

+ Create

☐ Sprint 3 15 Oct – 5 Nov (13 work items) 0 0 0 Complete sprint

☐ Sprint 4 5 Nov – 19 Nov (19 work items) 0 0 0 Start sprint



Epic
No epic
Login
UI: Op/User
Backend Process
Reporting
Notifications
Admin Console
Data & Integrations
DevOps & Reliability
P2P Prototype
QA & Release
+ Create epic

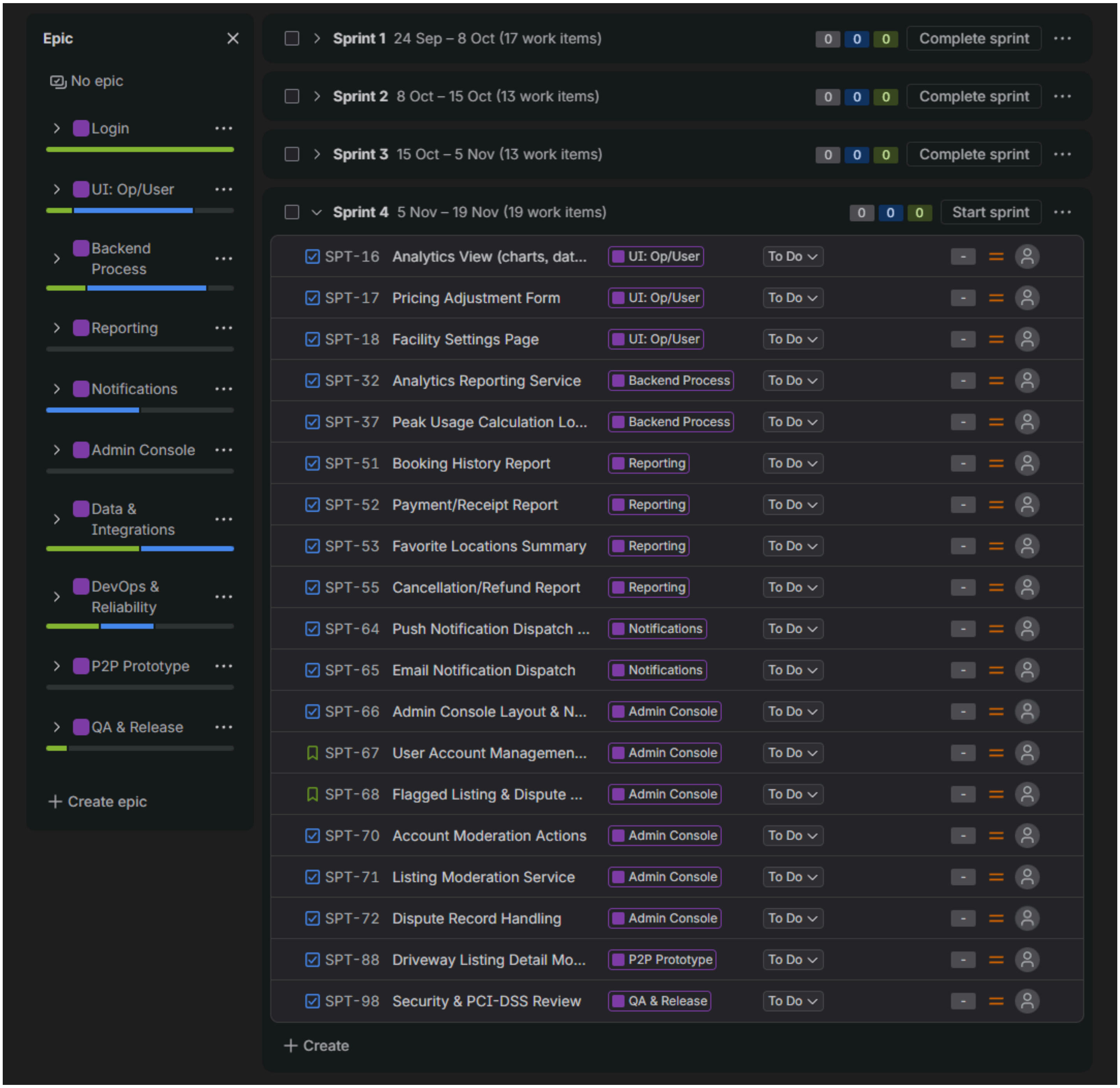
☐ **Sprint 1** 24 Sep – 8 Oct (17 work items) 0 0 0 Complete sprint

☐ **Sprint 2** 8 Oct – 22 Oct (13 work items) 0 0 0 Complete sprint

☐ **Sprint 3** 15 Oct – 5 Nov (13 work items) 0 0 0 Complete sprint

☑	SPT-14	Operator Dashboard Layou...	UI: Op/User	In Progress
☑	SPT-15	Occupancy Display Compo...	UI: Op/User	In Progress
☑	SPT-19	Operator Navigation Menu	UI: Op/User	In Progress
☑	SPT-20	Operator Dashboard Web L...	UI: Op/User	In Progress
☑	SPT-25	Reservation Management (...	UI: Op/User	In Progress
☑	SPT-26	Digital Pass Display (QR/ba...	UI: Op/User	In Progress
☑	SPT-31	Occupancy Aggregation S...	Backend Process	In Progress
☑	SPT-39	Payment Gateway Integrati...	Backend Process	In Progress
☑	SPT-40	Digital Pass Generation Ser...	Backend Process	In Progress
☑	SPT-42	Notification Service Backend	Backend Process	In Progress
	SPT-62	In-App Notification Display	Notifications	In Progress
	SPT-63	Notification Preference Set...	Notifications	In Progress
☑	SPT-83	Load Balancing & Horizont...	DevOps & Reliability	In Progress

☐ **Sprint 4** 5 Nov – 19 Nov (19 work items) 0 0 0 Start sprint



Epic

No epic

Login

UI: Op/User

Backend Process

Reporting

Notifications

Admin Console

Data & Integrations

DevOps & Reliability

P2P Prototype

QA & Release

+ Create epic

Sprint 1 24 Sep – 8 Oct (17 work items) 0 0 0 Complete sprint

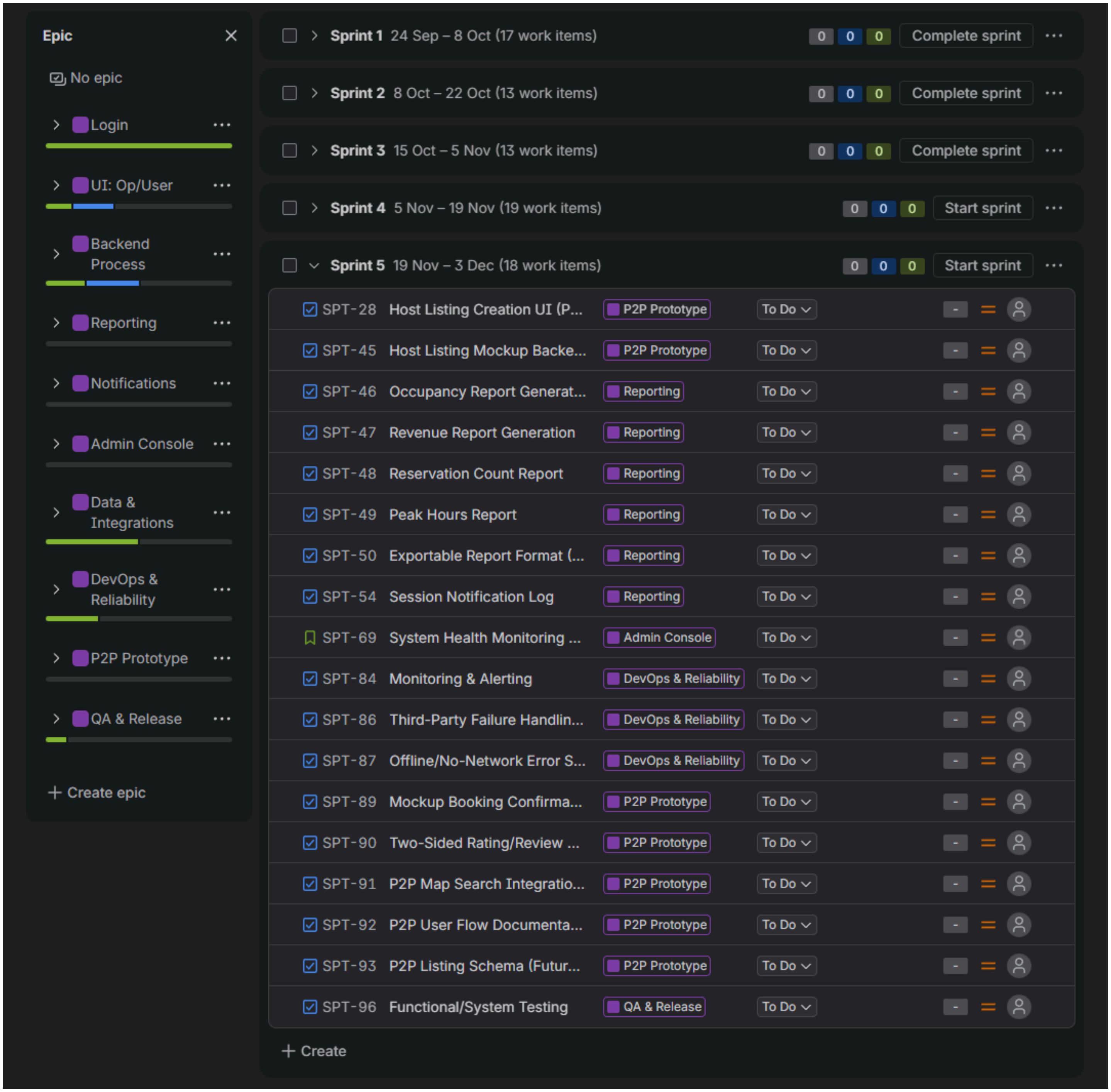
Sprint 2 8 Oct – 15 Oct (13 work items) 0 0 0 Complete sprint

Sprint 3 15 Oct – 5 Nov (13 work items) 0 0 0 Complete sprint

Sprint 4 5 Nov – 19 Nov (19 work items) 0 0 0 Start sprint

☑	SPT-16	Analytics View (charts, dat...	UI: Op/User	To Do
☑	SPT-17	Pricing Adjustment Form	UI: Op/User	To Do
☑	SPT-18	Facility Settings Page	UI: Op/User	To Do
☑	SPT-32	Analytics Reporting Service	Backend Process	To Do
☑	SPT-37	Peak Usage Calculation Lo...	Backend Process	To Do
☑	SPT-51	Booking History Report	Reporting	To Do
☑	SPT-52	Payment/Receipt Report	Reporting	To Do
☑	SPT-53	Favorite Locations Summary	Reporting	To Do
☑	SPT-55	Cancellation/Refund Report	Reporting	To Do
☑	SPT-64	Push Notification Dispatch ...	Notifications	To Do
☑	SPT-65	Email Notification Dispatch	Notifications	To Do
☑	SPT-66	Admin Console Layout & N...	Admin Console	To Do
	SPT-67	User Account Managemen...	Admin Console	To Do
	SPT-68	Flagged Listing & Dispute ...	Admin Console	To Do
☑	SPT-70	Account Moderation Actions	Admin Console	To Do
☑	SPT-71	Listing Moderation Service	Admin Console	To Do
☑	SPT-72	Dispute Record Handling	Admin Console	To Do
☑	SPT-88	Driveway Listing Detail Mo...	P2P Prototype	To Do
☑	SPT-98	Security & PCI-DSS Review	QA & Release	To Do

+ Create



Epic
No epic
Login
UI: Op/User
Backend Process
Reporting
Notifications
Admin Console
Data & Integrations
DevOps & Reliability
P2P Prototype
QA & Release
+ Create epic

☐ **Sprint 1** 24 Sep – 8 Oct (17 work items) 0 0 0 Complete sprint

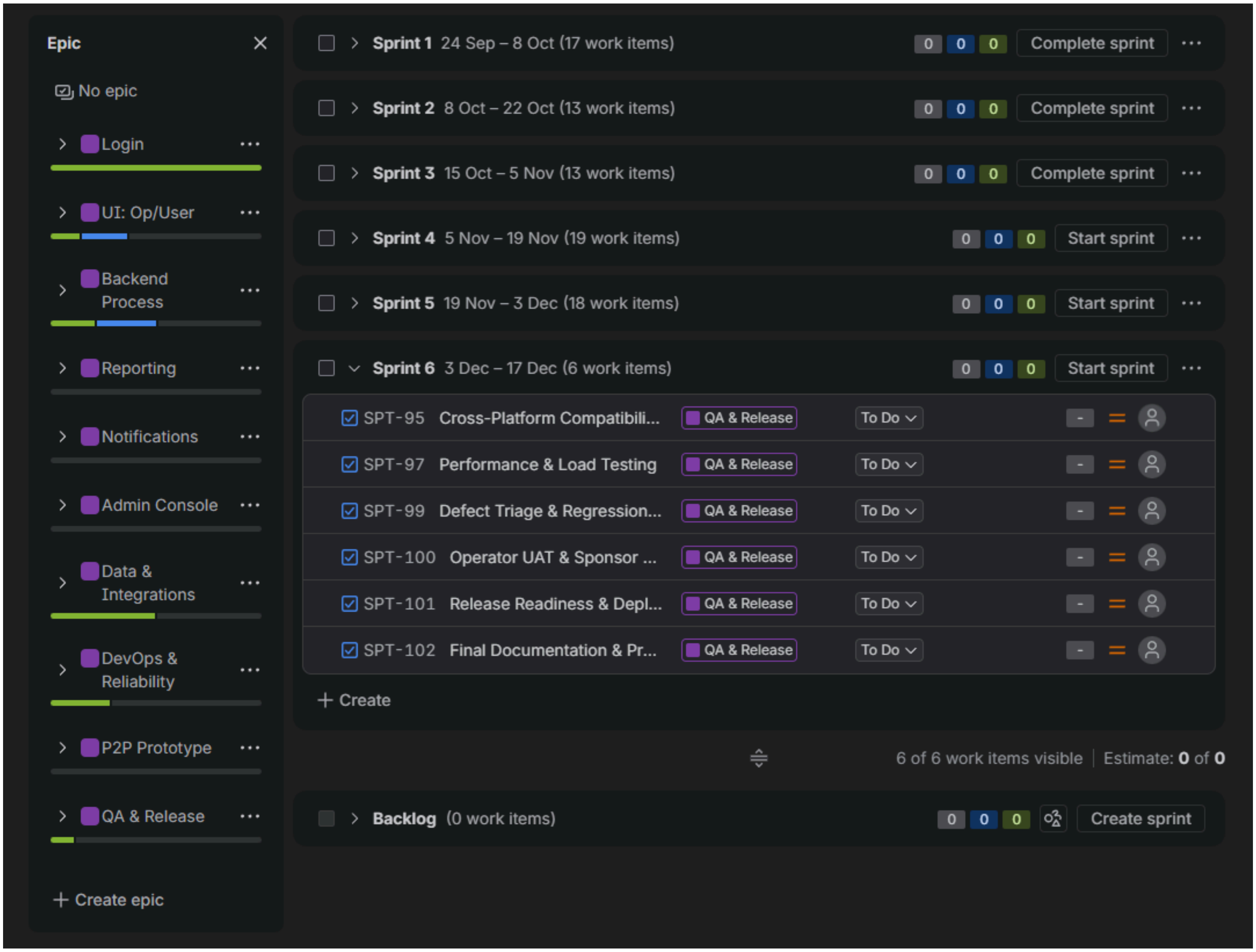
☐ **Sprint 2** 8 Oct – 22 Oct (13 work items) 0 0 0 Complete sprint

☐ **Sprint 3** 15 Oct – 5 Nov (13 work items) 0 0 0 Complete sprint

☐ **Sprint 4** 5 Nov – 19 Nov (19 work items) 0 0 0 Start sprint

☐ **Sprint 5** 19 Nov – 3 Dec (18 work items) 0 0 0 Start sprint

☑ SPT-28	Host Listing Creation UI (P...	P2P Prototype	To Do
☑ SPT-45	Host Listing Mockup Backe...	P2P Prototype	To Do
☑ SPT-46	Occupancy Report Generat...	Reporting	To Do
☑ SPT-47	Revenue Report Generation	Reporting	To Do
☑ SPT-48	Reservation Count Report	Reporting	To Do
☑ SPT-49	Peak Hours Report	Reporting	To Do
☑ SPT-50	Exportable Report Format (...	Reporting	To Do
☑ SPT-54	Session Notification Log	Reporting	To Do
SPT-69	System Health Monitoring ...	Admin Console	To Do
☑ SPT-84	Monitoring & Alerting	DevOps & Reliability	To Do
☑ SPT-86	Third-Party Failure Handlin...	DevOps & Reliability	To Do
☑ SPT-87	Offline/No-Network Error S...	DevOps & Reliability	To Do
☑ SPT-89	Mockup Booking Confirma...	P2P Prototype	To Do
☑ SPT-90	Two-Sided Rating/Review ...	P2P Prototype	To Do
☑ SPT-91	P2P Map Search Integratio...	P2P Prototype	To Do
☑ SPT-92	P2P User Flow Documenta...	P2P Prototype	To Do
☑ SPT-93	P2P Listing Schema (Futur...	P2P Prototype	To Do
☑ SPT-96	Functional/System Testing	QA & Release	To Do



Epic

No epic

- Login
- UI: Op/User
- Backend Process
- Reporting
- Notifications
- Admin Console
- Data & Integrations
- DevOps & Reliability
- P2P Prototype
- QA & Release

+ Create epic

☐ **Sprint 1** 24 Sep – 8 Oct (17 work items) 0 0 0 Complete sprint

☐ **Sprint 2** 8 Oct – 22 Oct (13 work items) 0 0 0 Complete sprint

☐ **Sprint 3** 15 Oct – 5 Nov (13 work items) 0 0 0 Complete sprint

☐ **Sprint 4** 5 Nov – 19 Nov (19 work items) 0 0 0 Start sprint

☐ **Sprint 5** 19 Nov – 3 Dec (18 work items) 0 0 0 Start sprint

☐ **Sprint 6** 3 Dec – 17 Dec (6 work items) 0 0 0 Start sprint

☑ SPT-95	Cross-Platform Compatibili...	QA & Release	To Do	-
☑ SPT-97	Performance & Load Testing	QA & Release	To Do	-
☑ SPT-99	Defect Triage & Regression...	QA & Release	To Do	-
☑ SPT-100	Operator UAT & Sponsor ...	QA & Release	To Do	-
☑ SPT-101	Release Readiness & Depl...	QA & Release	To Do	-
☑ SPT-102	Final Documentation & Pr...	QA & Release	To Do	-

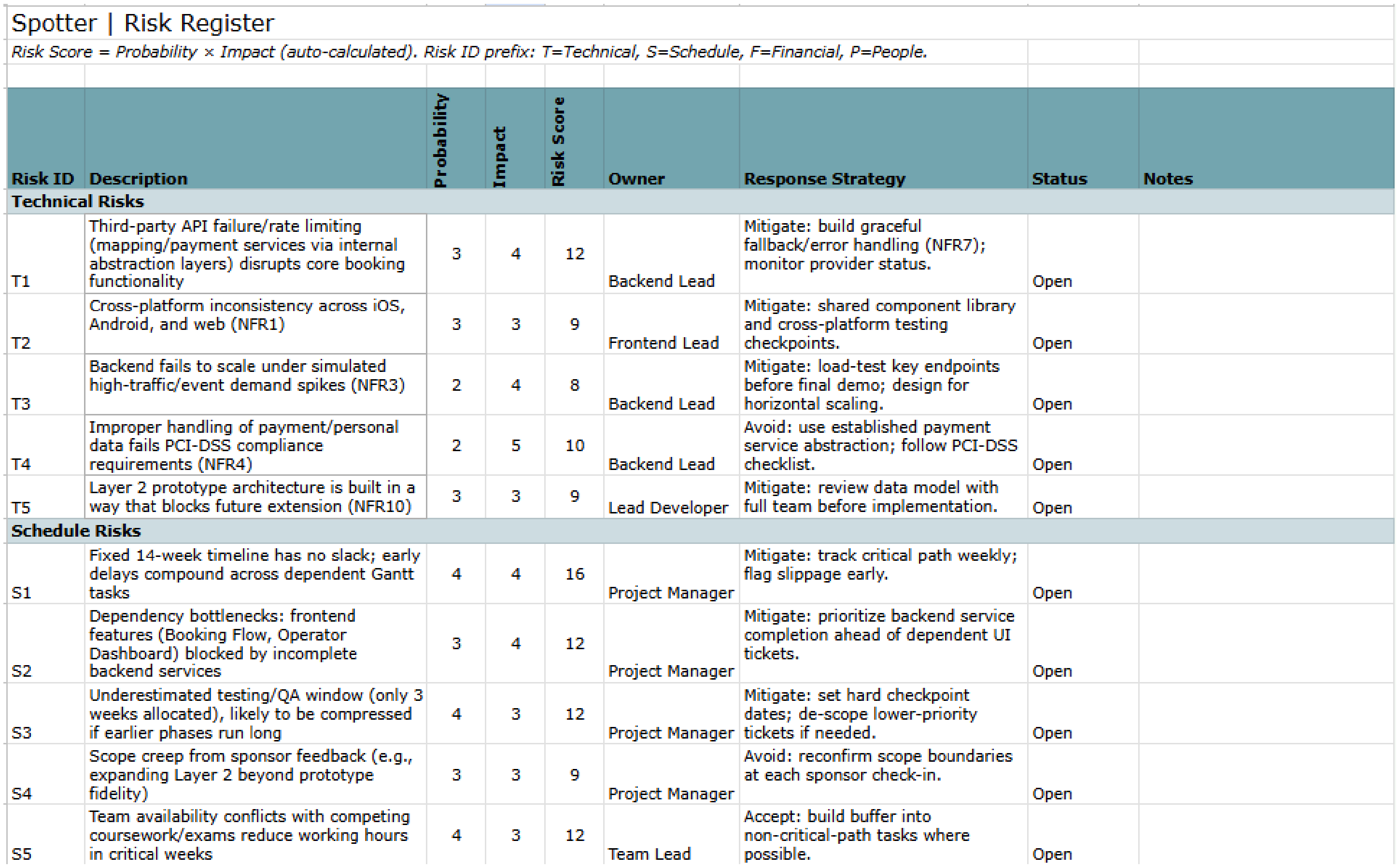
+ Create

6 of 6 work items visible | Estimate: **0** of **0**

☐ **Backlog** (0 work items) 0 0 0 Create sprint

9. Risk, Quality, and Communication Management

9.1 Risk Register



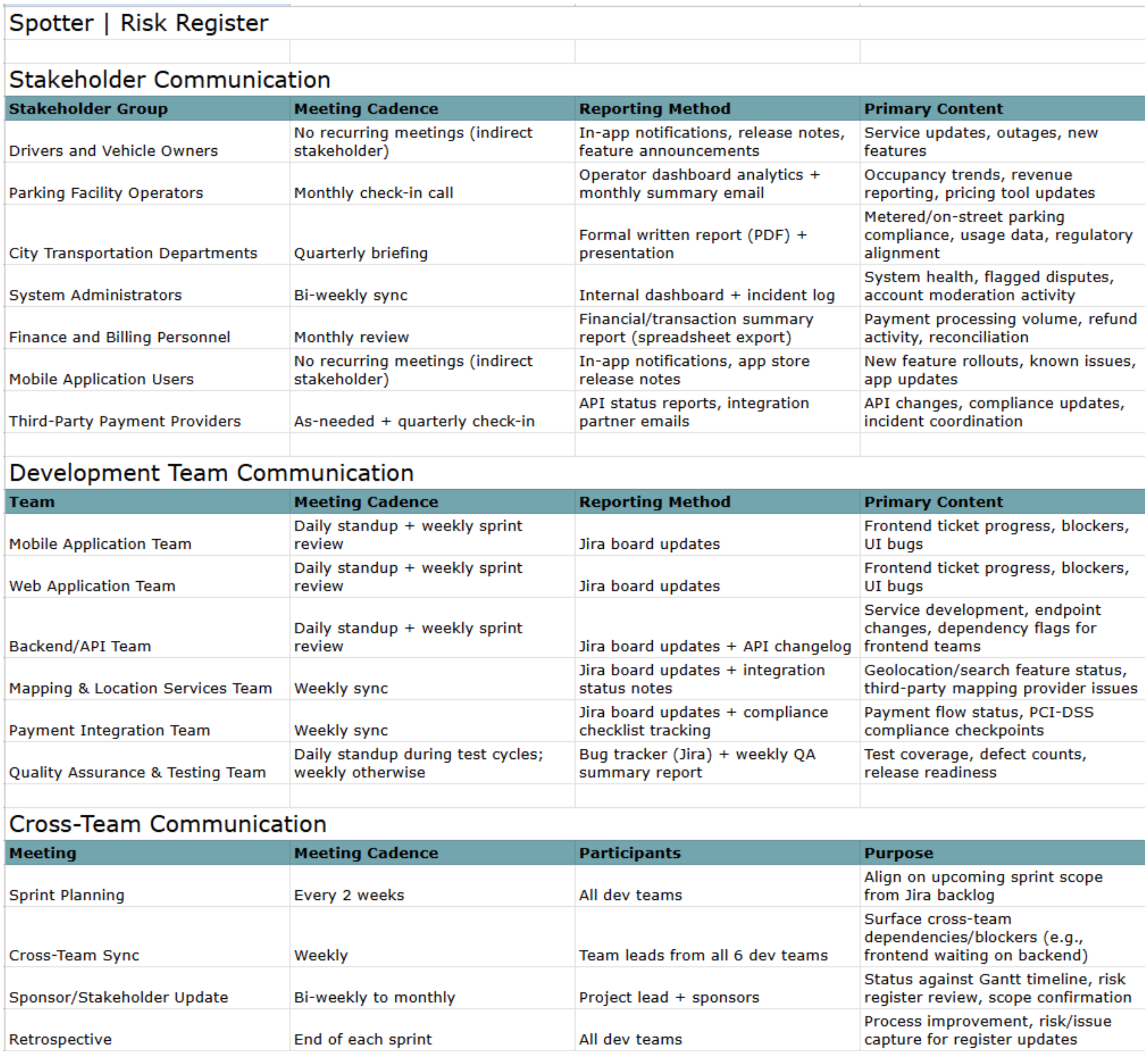
Spotter | Risk Register

Risk Score = Probability × Impact (auto-calculated). Risk ID prefix: T=Technical, S=Schedule, F=Financial, P=People.

Risk ID	Description	Probability	Impact	Risk Score	Owner	Response Strategy	Status	Notes
Technical Risks								
T1	Third-party API failure/rate limiting (mapping/payment services via internal abstraction layers) disrupts core booking functionality	3	4	12	Backend Lead	Mitigate: build graceful fallback/error handling (NFR7); monitor provider status.	Open	
T2	Cross-platform inconsistency across iOS, Android, and web (NFR1)	3	3	9	Frontend Lead	Mitigate: shared component library and cross-platform testing checkpoints.	Open	
T3	Backend fails to scale under simulated high-traffic/event demand spikes (NFR3)	2	4	8	Backend Lead	Mitigate: load-test key endpoints before final demo; design for horizontal scaling.	Open	
T4	Improper handling of payment/personal data fails PCI-DSS compliance requirements (NFR4)	2	5	10	Backend Lead	Avoid: use established payment service abstraction; follow PCI-DSS checklist.	Open	
T5	Layer 2 prototype architecture is built in a way that blocks future extension (NFR10)	3	3	9	Lead Developer	Mitigate: review data model with full team before implementation.	Open	
Schedule Risks								
S1	Fixed 14-week timeline has no slack; early delays compound across dependent Gantt tasks	4	4	16	Project Manager	Mitigate: track critical path weekly; flag slippage early.	Open	
S2	Dependency bottlenecks: frontend features (Booking Flow, Operator Dashboard) blocked by incomplete backend services	3	4	12	Project Manager	Mitigate: prioritize backend service completion ahead of dependent UI tickets.	Open	
S3	Underestimated testing/QA window (only 3 weeks allocated), likely to be compressed if earlier phases run long	4	3	12	Project Manager	Mitigate: set hard checkpoint dates; de-scope lower-priority tickets if needed.	Open	
S4	Scope creep from sponsor feedback (e.g., expanding Layer 2 beyond prototype fidelity)	3	3	9	Project Manager	Avoid: reconfirm scope boundaries at each sponsor check-in.	Open	
S5	Team availability conflicts with competing coursework/exams reduce working hours in critical weeks	4	3	12	Team Lead	Accept: build buffer into non-critical-path tasks where possible.	Open	

Financial Risks								
F1	Third-party service costs (mapping/payment APIs) exceed current no-budget-constraint assumption if ever deployed	2	2	4	Sponsor Liaison	Accept: documented as an assumption; revisit if moving beyond prototype.	Open	
F2	Cost of compliance infrastructure (PCI-DSS, data privacy tooling/certification) not yet budgeted	2	3	6	Sponsor Liaison	Accept: out of scope for current prototype; note for future-phase planning.	Open	
F3	Needed software/design/testing tool licenses exceed assumed available budget	2	2	4	Project Manager	Mitigate: prioritize free/open-source tooling where feasible.	Open	
F4	Hidden cost of rework from technical risks (e.g., Layer 2 architecture lock-in) consumes team time/effort	3	3	9	Lead Developer	Mitigate: design review before implementation to reduce rework risk.	Open	
F5	Future-phase cost of full Layer 2 functionality (verification, payouts, insurance) underestimated	3	2	6	Sponsor Liaison	Accept: explicitly flagged as future-phase/out of current scope.	Open	
People Risks								
P1	Uneven skill distribution (e.g., payment integration, map rendering) creates single points of failure	3	4	12	Team Lead	Mitigate: pair less-experienced members with relevant component owners.	Open	
P2	Team member attrition or reduced availability removes a contributor mid-project	2	4	8	Team Lead	Mitigate: maintain shared documentation so others can pick up tickets.	Open	
P3	Miscommunication across frontend/backend ownership causes duplicate work or integration mismatches	3	3	9	Team Lead	Mitigate: maintain clear ownership via Jira board and regular sync-ups.	Open	
P4	Knowledge concentration risk: one team member solely understands a critical system (e.g., reservation logic)	3	4	12	Team Lead	Mitigate: require code documentation and periodic knowledge-sharing sessions.	Open	
P5	Stakeholder/sponsor misalignment on what 'prototype-level' Layer 2 actually delivers	3	3	9	Project Manager	Avoid: clarify prototype-fidelity definition explicitly in sponsor updates.	Open	

9.2 Communication Plan



Spotter | Risk Register

Stakeholder Communication

Stakeholder Group	Meeting Cadence	Reporting Method	Primary Content
Drivers and Vehicle Owners	No recurring meetings (indirect stakeholder)	In-app notifications, release notes, feature announcements	Service updates, outages, new features
Parking Facility Operators	Monthly check-in call	Operator dashboard analytics + monthly summary email	Occupancy trends, revenue reporting, pricing tool updates
City Transportation Departments	Quarterly briefing	Formal written report (PDF) + presentation	Metered/on-street parking compliance, usage data, regulatory alignment
System Administrators	Bi-weekly sync	Internal dashboard + incident log	System health, flagged disputes, account moderation activity
Finance and Billing Personnel	Monthly review	Financial/transaction summary report (spreadsheet export)	Payment processing volume, refund activity, reconciliation
Mobile Application Users	No recurring meetings (indirect stakeholder)	In-app notifications, app store release notes	New feature rollouts, known issues, app updates
Third-Party Payment Providers	As-needed + quarterly check-in	API status reports, integration partner emails	API changes, compliance updates, incident coordination

Development Team Communication

Team	Meeting Cadence	Reporting Method	Primary Content
Mobile Application Team	Daily standup + weekly sprint review	Jira board updates	Frontend ticket progress, blockers, UI bugs
Web Application Team	Daily standup + weekly sprint review	Jira board updates	Frontend ticket progress, blockers, UI bugs
Backend/API Team	Daily standup + weekly sprint review	Jira board updates + API changelog	Service development, endpoint changes, dependency flags for frontend teams
Mapping & Location Services Team	Weekly sync	Jira board updates + integration status notes	Geolocation/search feature status, third-party mapping provider issues
Payment Integration Team	Weekly sync	Jira board updates + compliance checklist tracking	Payment flow status, PCI-DSS compliance checkpoints
Quality Assurance & Testing Team	Daily standup during test cycles; weekly otherwise	Bug tracker (Jira) + weekly QA summary report	Test coverage, defect counts, release readiness

Cross-Team Communication

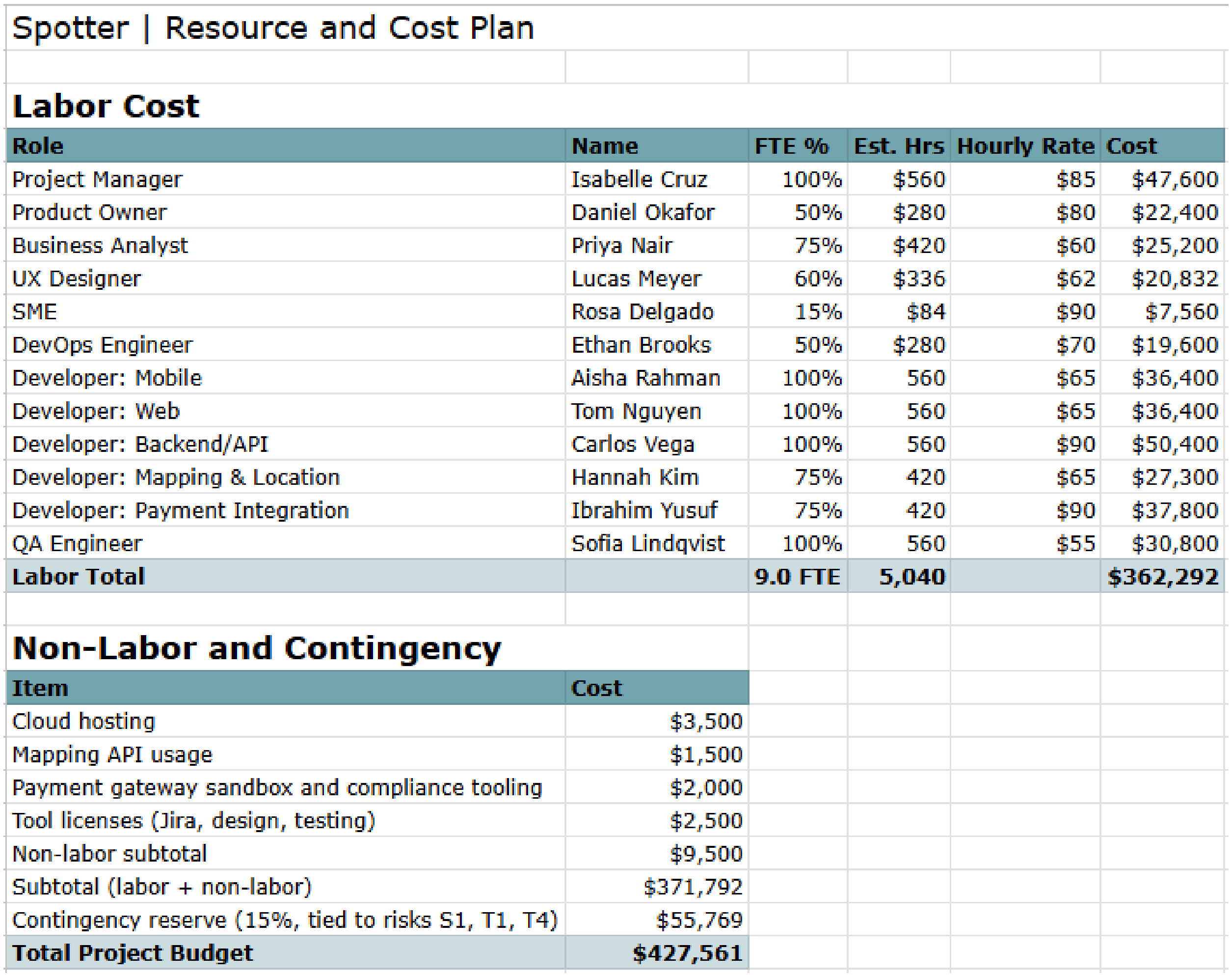
Meeting	Meeting Cadence	Participants	Purpose
Sprint Planning	Every 2 weeks	All dev teams	Align on upcoming sprint scope from Jira backlog
Cross-Team Sync	Weekly	Team leads from all 6 dev teams	Surface cross-team dependencies/blockers (e.g., frontend waiting on backend)
Sponsor/Stakeholder Update	Bi-weekly to monthly	Project lead + sponsors	Status against Gantt timeline, risk register review, scope confirmation
Retrospective	End of each sprint	All dev teams	Process improvement, risk/issue capture for register updates

10. Roles and Resources

10.1 Team Roles and Responsibilities

Role	Name	Skill Level	Allocation	Focus
Project Manager (PM)	Isabelle Cruz	Senior	100%	Schedule (Gantt), risk register, sponsor updates, cross-team coordination
Product Owner (PO)	Daniel Okafor	Senior	50%	Owns the Jira backlog, prioritizes tickets, accepts completed work, represents sponsor interests
Business Analyst (BA)	Priya Nair	Mid	75%	Maintains the SRS, writes use cases and acceptance criteria, traces requirements to tickets
UX Designer	Lucas Meyer	Mid	60%	Wireframes, user flows, Layer 2 mockups, operator dashboard layout
Subject Matter Expert (SME)	Rosa Delgado	Senior	15%	Parking operations and municipal parking regulations; validates pricing, occupancy, and metered-parking logic
DevOps Engineer	Ethan Brooks	Mid	50%	Cloud hosting, CI/CD, load-balanced scaling (NFR3), monitoring and logging
Developer: Mobile	Aisha Rahman	Mid	100%	iOS/Android UI, booking flow, digital pass display
Developer: Web	Tom Nguyen	Mid	100%	Web client, operator dashboard, admin console
Developer: Backend/API	Carlos Vega	Senior	100%	Reservation, search, and inventory services, data layer, RBAC
Developer: Mapping & Location	Hannah Kim	Mid	75%	Map rendering, geolocation, search autocomplete, mapping integration
Developer: Payment Integration	Ibrahim Yusuf	Senior	75%	Payment flow, refunds, receipts, PCI-DSS compliance checkpoints
QA Engineer	Sofia Lindqvist	Mid	100%	Test plans, cross-platform testing, defect tracking, release readiness

10.2 Resource and Cost Plan



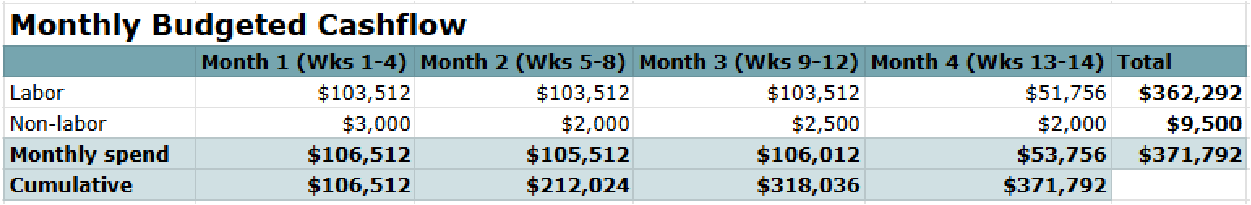
Spotter | Resource and Cost Plan

Labor Cost

Role	Name	FTE %	Est. Hrs	Hourly Rate	Cost
Project Manager	Isabelle Cruz	100%	$560	$85	$47,600
Product Owner	Daniel Okafor	50%	$280	$80	$22,400
Business Analyst	Priya Nair	75%	$420	$60	$25,200
UX Designer	Lucas Meyer	60%	$336	$62	$20,832
SME	Rosa Delgado	15%	$84	$90	$7,560
DevOps Engineer	Ethan Brooks	50%	$280	$70	$19,600
Developer: Mobile	Aisha Rahman	100%	560	$65	$36,400
Developer: Web	Tom Nguyen	100%	560	$65	$36,400
Developer: Backend/API	Carlos Vega	100%	560	$90	$50,400
Developer: Mapping & Location	Hannah Kim	75%	420	$65	$27,300
Developer: Payment Integration	Ibrahim Yusuf	75%	420	$90	$37,800
QA Engineer	Sofia Lindqvist	100%	560	$55	$30,800
Labor Total		**9.0 FTE**	**5,040**		**$362,292**

Non-Labor and Contingency

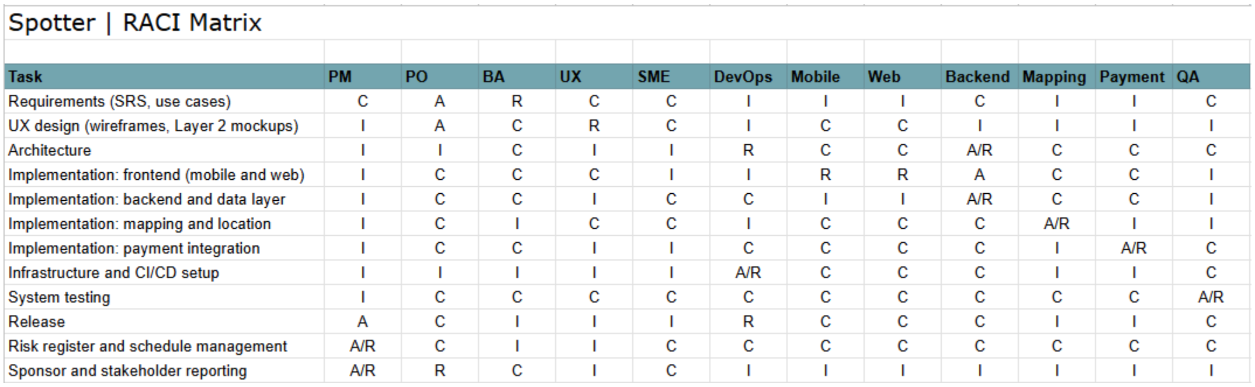
Item	Cost
Cloud hosting	$3,500
Mapping API usage	$1,500
Payment gateway sandbox and compliance tooling	$2,000
Tool licenses (Jira, design, testing)	$2,500
Non-labor subtotal	$9,500
Subtotal (labor + non-labor)	$371,792
Contingency reserve (15%, tied to risks S1, T1, T4)	$55,769
Total Project Budget	**$427,561**



Monthly Budgeted Cashflow

	Month 1 (Wks 1-4)	Month 2 (Wks 5-8)	Month 3 (Wks 9-12)	Month 4 (Wks 13-14)	Total
Labor	$103,512	$103,512	$103,512	$51,756	**$362,292**
Non-labor	$3,000	$2,000	$2,500	$2,000	**$9,500**
Monthly spend	**$106,512**	**$105,512**	**$106,012**	**$53,756**	**$371,792**
Cumulative	**$106,512**	**$212,024**	**$318,036**	**$371,792**	

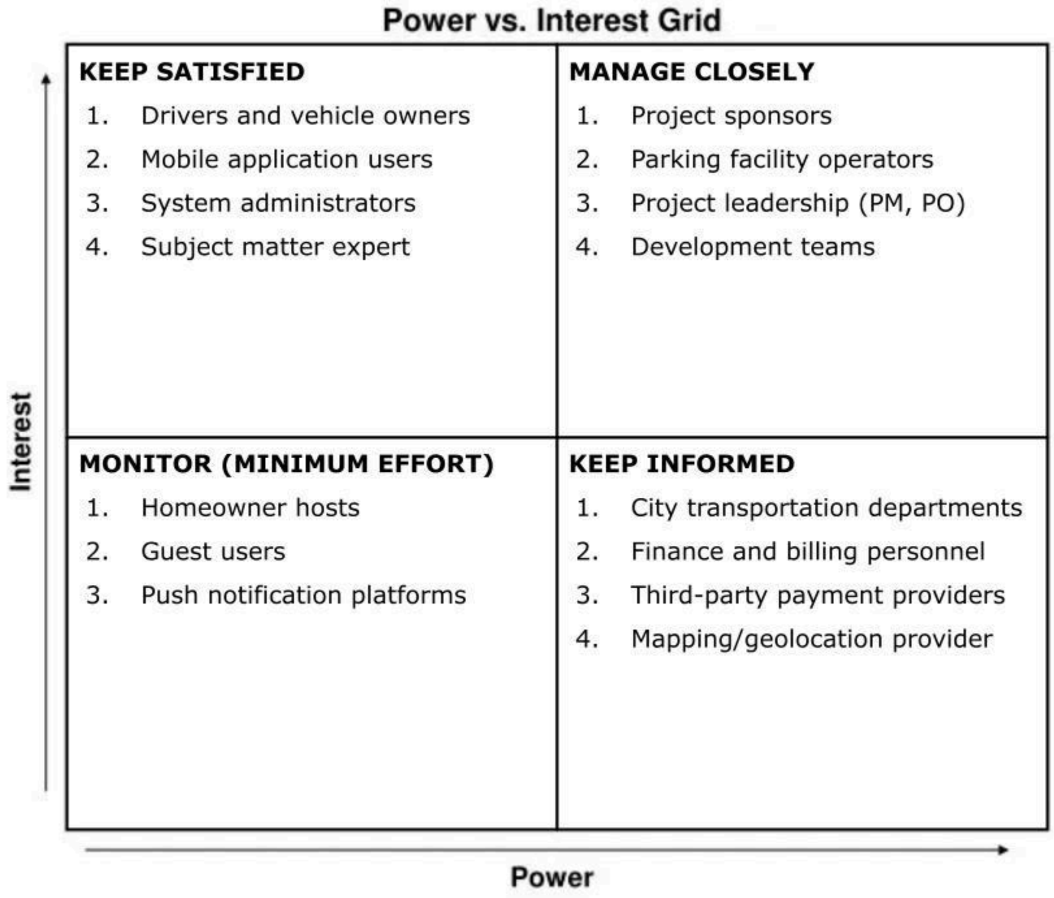
10.3 RACI Matrix



Spotter | RACI Matrix

Task	PM	PO	BA	UX	SME	DevOps	Mobile	Web	Backend	Mapping	Payment	QA
Requirements (SRS, use cases)	C	A	R	C	C	I	I	I	C	I	I	C
UX design (wireframes, Layer 2 mockups)	I	A	C	R	C	I	C	C	I	I	I	I
Architecture	I	I	C	I	I	R	C	C	A/R	C	C	C
Implementation: frontend (mobile and web)	I	C	C	C	I	I	R	R	A	C	C	I
Implementation: backend and data layer	I	C	C	I	C	C	I	I	A/R	C	C	I
Implementation: mapping and location	I	C	I	C	C	I	C	C	C	A/R	I	I
Implementation: payment integration	I	C	C	I	I	C	C	C	C	I	A/R	C
Infrastructure and CI/CD setup	I	I	I	I	I	A/R	C	C	C	I	I	C
System testing	I	C	C	C	C	C	C	C	C	C	C	A/R
Release	A	C	I	I	I	R	C	C	C	I	I	C
Risk register and schedule management	A/R	C	I	I	C	C	C	C	C	C	C	C
Sponsor and stakeholder reporting	A/R	R	C	I	C	I	I	I	I	I	I	I

11. Power-Interest Grid


Power vs. Interest Grid

KEEP SATISFIED

1. Drivers and vehicle owners
2. Mobile application users
3. System administrators
4. Subject matter expert

MANAGE CLOSELY

1. Project sponsors
2. Parking facility operators
3. Project leadership (PM, PO)
4. Development teams

MONITOR (MINIMUM EFFORT)

1. Homeowner hosts
2. Guest users
3. Push notification platforms

KEEP INFORMED

1. City transportation departments
2. Finance and billing personnel
3. Third-party payment providers
4. Mapping/geolocation provider